Algorithmics and Data Structures

Session 5
Greedy Algorithms

Ali Ismail-Fawaz

ENSISA, Université Haute-Alsace

Overview

- **What is a Greedy Algorithm?**
  - Core idea: make the locally optimal choice at each step.
  - When this strategy works and when it fails.
- **Key Concepts**
  - Greedy-choice property
  - Optimal substructure
- **Easy real-life examples**
- **First simple problem**
- **Why it can work**
- **When it can fail**

Contents

What is a Greedy Algorithm

Easy Problems

Medium Problems

Advanced Problems

What is a Greedy Algorithm?

Make the best choice right now!

1. Local Decision

Choose the option that looks best at the current step.

2. No Backtracking

Once a decision is made, we do not reconsider it.

3. No Guarantee

The best choice **now** may not lead to the best solution **overall**.

Example: Making Change



© OpenAI

Always pick the largest coin that does not exceed the remaining amount.

Does Greedy Always Work?

Goal: Make change using the minimum number of coins.
Assumption: An unlimited number of coins is available for each denomination.

Case A: Coins $\{1, 5, 10\}$

Target: 18

Case B: Coins $\{1, 3, 4\}$

Target: 6

Does Greedy Always Work?

Goal: Make change using the minimum number of coins.
Assumption: An unlimited number of coins is available for each denomination.

Case A: Coins $\{1, 5, 10\}$

Target: 18
Greedy picks:

$$10 + 5 + 1 + 1 + 1$$

5 coins

Case B: Coins $\{1, 3, 4\}$

Target: 6

Does Greedy Always Work?

Goal: Make change using the minimum number of coins.
Assumption: An unlimited number of coins is available for each denomination.

Case A: Coins $\{1, 5, 10\}$

Target: 18
Greedy picks:

$$10 + 5 + 1 + 1 + 1$$

5 coins
Optimal!

Case B: Coins $\{1, 3, 4\}$

Target: 6

Does Greedy Always Work?

Goal: Make change using the minimum number of coins.
Assumption: An unlimited number of coins is available for each denomination.

Case A: Coins $\{1, 5, 10\}$

Target: 18
Greedy picks:

$$10 + 5 + 1 + 1 + 1$$

5 coins
Optimal!

Case B: Coins $\{1, 3, 4\}$

Target: 6
Greedy picks:

$$4 + 1 + 1$$

3 coins

Does Greedy Always Work?

Goal: Make change using the minimum number of coins.
Assumption: An unlimited number of coins is available for each denomination.

Case A: Coins $\{1, 5, 10\}$

Target: 18
Greedy picks:

$$10 + 5 + 1 + 1 + 1$$

5 coins
Optimal!

Case B: Coins $\{1, 3, 4\}$

Target: 6
Greedy picks:

$$4 + 1 + 1$$

3 coins
But we could choose: $3 + 3$
Only 2 coins!

Does Greedy Always Work?

Goal: Make change using the minimum number of coins.
Assumption: An unlimited number of coins is available for each denomination.

Case A: Coins $\{1, 5, 10\}$

Target: 18
Greedy picks:

$$10 + 5 + 1 + 1 + 1$$

5 coins
Optimal!

Case B: Coins $\{1, 3, 4\}$

Target: 6
Greedy picks:

$$4 + 1 + 1$$

3 coins
But we could choose: $3 + 3$
Only 2 coins!

Key Takeaway

A locally optimal choice does not necessarily lead to a **globally optimal solution**.

When Can We Trust Greedy?

Greedy algorithms produce optimal solutions when the problem has two important properties.

1. Greedy-Choice Property

There exists an optimal solution that starts with the **locally optimal choice**.

Making this choice does not prevent us from reaching an optimal solution.

2. Optimal Substructure

After making a choice, the remaining problem can be solved optimally.

An optimal solution contains optimal solutions to its remaining subproblems.

Examples

Activity Selection • Huffman Coding • Minimum Spanning Tree

Contents

What is a Greedy Algorithm

Easy Problems

Medium Problems

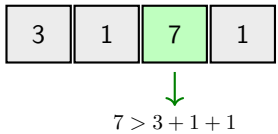
Advanced Problems

Smallest Subset Sum Greater Than All Others

Problem Statement

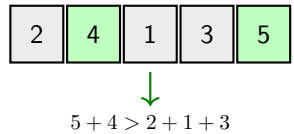
Given an array of **non-negative integers**, find a subset containing the **fewest elements** such that its sum is **strictly greater** than the sum of the remaining elements.

Example 1: $[3, 1, 7, 1]$



Answer: 1 element

Example 2: $[2, 4, 1, 3, 5]$



Answer: 2 elements

Objective

Minimize the **number of selected elements**, not their sum!

Real-World Application: Budgeting

Pick the few largest expenses whose total is more than half of the total budget.

Naive Solution: Brute Force

Idea

Enumerate **all possible subsets**, check whether each satisfies the condition, and keep the valid subset with the **fewest elements**.

Example: $A = [3, 1, 7]$

Subset	Valid?
$\emptyset$	No
$\{3\}$	No
$\{1\}$	No
$\{7\}$	**Yes!**
$\{3, 1\}$	No
$\{3, 7\}$	Yes
$\{1, 7\}$	Yes
$\{3, 1, 7\}$	Yes

Time Complexity $\mathcal{O}(n \cdot 2^n)$

- Each element is either **selected or not**: 2^n subsets.
- Checking each subset requires up to n operations.

Space Complexity $\mathcal{O}(n)$

- Generate subsets **one at a time**.
- Store at most n elements (current subset / recursion).

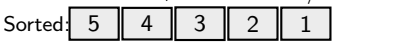
Optimal Solution: Greedy

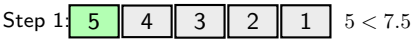
Greedy Strategy

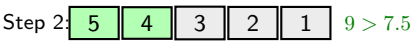
Sort the elements in **descending order**, then repeatedly select the largest remaining element until the selected sum exceeds **half the total sum**.

Example: $A = [2, 4, 1, 3, 5]$

Total: $S = 15$, threshold: $S/2 = 7.5$







Stop! Selected: $\{5, 4\}$
Only 2 elements

Time Complexity $\mathcal{O}(n \log n)$

- Compute total sum: $\mathcal{O}(n)$.
- Sort elements in descending order: $\mathcal{O}(n \log n)$.
- Select elements until threshold: at most n steps, $\mathcal{O}(n)$.

Space Complexity $\mathcal{O}(\log n)$

- Quicksort: $\mathcal{O}(\log n)$ space.

Why Greedy Works

The k largest elements give the **maximum sum** possible with k elements.
$\sum_{i=1}^{k} a_i \leq \frac{S}{2} \Rightarrow$ No solution with k elements Therefore, the first successful k is **optimal**.

Smallest Subset Sum Greater Than All Others

Greedy Algorithm

`Input: Array A, size n`
`Output: size of smallest subset with bigger sum than all others`
`Instructions:`

1. $total \leftarrow sum(A)$
2. `sort` A `in descending order`
3. $count \leftarrow 0$
4. $current_total \leftarrow 0$
5. `For i from 1 to n do`
   a. $current_total \leftarrow current_total + A[i]$
   b. $count \leftarrow count + 1$
   c. `If` $current_total > total - current_total$
      `Return` $count$
6. `Return` $count$

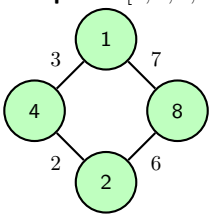
Maximize Sum of Consecutive Differences (Circular Array)

Problem Statement

Given an array of integers, **rearrange its elements** to **maximize the sum of absolute differences** between consecutive elements, including the last and first.

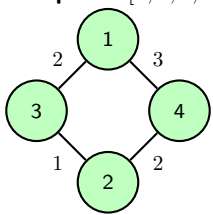
$$D = \sum_{i=1}^{n} |a_i - a_{i+1}|, \qquad a_{n+1} = a_1$$

Example 1: $[4, 2, 1, 8]$



Maximum: $7 + 6 + 2 + 3 = 18$

Example 2: $[1, 2, 3, 4]$



Maximum: $3 + 2 + 1 + 2 = 8$

Objective

Find the arrangement with the **largest total difference**.

Remember: the array is **circular**!

The last and first elements are also neighbors.

Real-World Application: Sensor Placement

Arrange sensors with different readings around a circular track to maximize variation inter neighbors.

Naive Solution: Brute Force

Idea

Generate **all possible permutations**, calculate the circular difference sum for each, and keep the arrangement with the **maximum sum**.

Example: $A = [1, 2, 3, 4]$

Permutation	Sum
$[1, 2, 3, 4]$	6
$[1, 2, 4, 3]$	6
$[1, 3, 2, 4]$	**8**
$[1, 3, 4, 2]$	6
$[1, 4, 2, 3]$	**8**
$\vdots$	
$[4, 3, 2, 1]$	6

$4! = 24$ permutations

Time Complexity $\mathcal{O}(n \cdot n!)$

- There are $n!$ possible permutations.
- Computing each circular sum requires n operations.

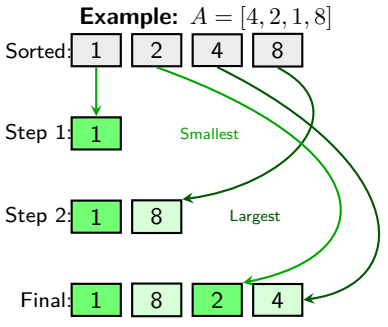
Space Complexity $\mathcal{O}(n)$

- Generate permutations **one at a time**.
- Store at most n elements (permutation / recursion).

Optimal Solution: Greedy

Greedy Strategy

Sort the array in **ascending order**, then **alternate between the smallest and largest** remaining elements to maximize adjacent differences.



Maximum difference sum
$7 + 6 + 2 + 3 = \mathbf{18}$

Time Complexity $\mathcal{O}(n \log n)$

- Sort the array: $\mathcal{O}(n \log n)$.
- Alternate smallest/largest: $\mathcal{O}(n)$.
- Compute circular sum: $\mathcal{O}(n)$.

Space Complexity $\mathcal{O}(n)$

- Store the rearranged array of n elements.
- Could be $\mathcal{O}(1)$ if done in-place

Why Greedy Works

Large differences are obtained by placing **small and large elements next to each other**.

Maximize Sum of Consecutive Differences (Circular Array)

Greedy Algorithm

```
Input: Array A, size n
Output: Maximize Sum of Consecutive Differences
Instructions:
```

1. sort A in increasing order
2. $sum \leftarrow 0$
3. For i from 1 to $\lfloor n/2 \rfloor$ do
   a. $sum \leftarrow sum + 2 \cdot (A[n+1-i] - A[i])$
4. Return sum

Maximize Circular Differences – Why Does Greedy Work? (1/2)

Let $a_1 \leq a_2 \leq \cdots \leq a_n$ be the sorted array.

Observation: Every Element Has Two Neighbors

Consider three consecutive elements x, a_i, y. The contribution involving a_i is:

$$|x - a_i| + |a_i - y|$$

- If a_i is **smaller than both neighbors**:

$$(x - a_i) + (y - a_i) = x + y - \mathbf{2a_i}$$

- If a_i is **larger than both neighbors**:

$$(a_i - x) + (a_i - y) = \mathbf{2a_i} - x - y$$

- If a_i is **between its neighbors**:

$$(a_i - x) + (y - a_i) = y - x \quad \Rightarrow \quad \textbf{contribution of } a_i = 0$$

What Does This Tell Us?

Each element has a coefficient of -2, 0, or $+2$.
Since every difference contributes one positive and one negative term, the coefficients must sum to zero.

To maximize the sum, we want the **largest values with coefficient** $+2$ and the **smallest values with coefficient** -2.

Maximize Circular Differences – Why Does Greedy Work? (2/2)

Let $k = \lfloor n/2 \rfloor$.

The Maximum Possible Value

A circular array cannot have more than k strict local maxima: two local maxima cannot be adjacent. Similarly, it cannot have more than k strict local minima. Therefore, the sum cannot exceed:

$$D \leq 2 \left(\underbrace{\sum_{i=n-k+1}^{n} a_i}_{\text{Largest } k} - \underbrace{\sum_{i=1}^{k} a_i}_{\text{Smallest } k} \right)$$

Can We Actually Reach This Bound?

Yes! Alternate the smallest and largest remaining elements: $a_1,\ a_n,\ a_2,\ a_{n-1},\ a_3,\ a_{n-2}, \ldots$

- **Even** $n = 2k$**:** Every small element is surrounded by large elements, and vice versa.
- **Odd** $n = 2k+1$**:** The middle element a_{k+1} has one smaller and one larger neighbor, so its coefficient is 0.

Thus, the arrangement achieves the upper bound.

Conclusion

Since the upper bound is achievable, it is the maximum: $\boxed{D_{\max} = 2 \sum_{i=1}^{\lfloor n/2 \rfloor} (a_{n+1-i} - a_i)}$

Contents

What is a Greedy Algorithm

Easy Problems

Medium Problems

Advanced Problems

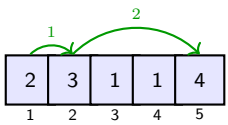
Minimum Number of Jumps to Reach End

Problem Statement

Given an array A of non-negative integers, where $A[i]$ represents the **maximum jump length** from index i, find the **minimum number of jumps** required to reach the last index n.

You may jump any distance from 1 to $A[i]$. Return -1 if the last index is unreachable.

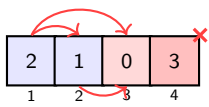
Example 1: $[2, 3, 1, 1, 4]$



Minimum: 2 **jumps**

Path: $1 \to 2 \to 5$

Example 2: $[2, 1, 0, 3]$



Result: -1

All paths stop at index 3.

Objective

Reach the **last index** using the **fewest jumps**.

Each element specifies the **maximum distance** you can jump.

Return -1 if the destination is **unreachable**.

Real-World Application: Communication Networks

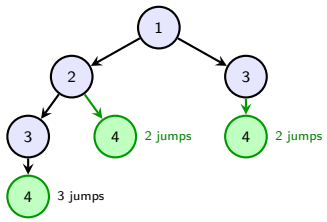
Minimize the number of relay hops needed to transmit a signal along a network, where each router has a limited transmission range.

Minimum Number of Jumps to Reach End – Naive Solution

Brute-Force Approach

Starting from index 1, recursively explore **all possible jumps**. Return the **minimum number of jumps** among paths reaching index n, or -1 if none exists.

Example: $A = [2, 2, 1, 1]$



Explored Paths

$1 \to 2 \to 3 \to 4$

$1 \to 2 \to 4$

$1 \to 3 \to 4$

Result

Minimum: 2 jumps

Time Complexity

$\boxed{\mathcal{O}(2^n)}$

Exponentially many possible paths.

Space Complexity

$\boxed{\mathcal{O}(n)}$

Recursive call stack.

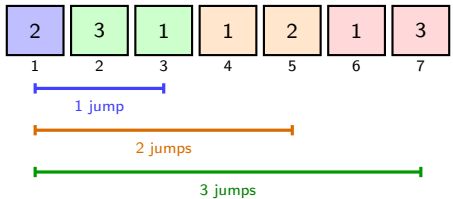
Minimum Number of Jumps to Reach End – Greedy Strategy

Greedy Approach

At each step, examine **all indices reachable** with the current number of jumps.

Extend the reachable range as far as possible. Each extension corresponds to **one additional jump**.

Example: $A = [2, 3, 1, 1, 2, 1, 3]$



Jump 1

From index 1:

$$1 + A[1] = 3$$

Reach indices 1–3.

Jump 2

Examine indices 2–3:

$$\max(2 + 3, 3 + 1) = 5$$

Reach indices 1–5.

Jump 3

Examine indices 4–5:

$$\max(4 + 1, 5 + 2) = 7$$

Destination reached!

Minimum Number of Jumps to Reach End – Why Greedy Works

Key Idea

The algorithm explores positions **level by level**, where each level corresponds to a number of jumps.
It computes the **farthest reachable index** before moving to the next level.

Three Variables

jumps
Number of jumps needed to reach the current range.

currentReach
Last index reachable with `jumps` jumps.

maxReach
Farthest index reachable with one additional jump.

Greedy Decision

For each index i, update:

$$maxReach \leftarrow \max(maxReach,\ i + A[i])$$

When $i = currentReach$:

We have examined **all positions** reachable with the current number of jumps.

To reach any position beyond this range, we **must take another jump**:

$$\boxed{\begin{aligned} jumps &\leftarrow jumps + 1 \\ currentReach &\leftarrow maxReach \end{aligned}}$$

If $maxReach = currentReach < n$, the destination is unreachable.

Why Is This Optimal? & Complexity

Optimality: Each range contains **all positions reachable** with at most the current number of jumps. Therefore, when the destination first becomes reachable, **no solution with fewer jumps exists**.

Time: $\boxed{\mathcal{O}(n)}$ Each index is examined at most once. **Space:** $\boxed{\mathcal{O}(1)}$ Only three variables are maintained.

Minimum Number of Jumps to Reach End

Greedy Algorithm

```
Input: Array A, size n
Output: Minimum Number of Jumps to Reach End
Instructions:
```

1. If $n == 1$
   Return 0
2. If $A[1] == 0$
   Return -1
3. $jumps$, $maxReach$, $currentReach \leftarrow 0$, 1, 1
4. For i from 1 to $n-1$ do
   a. $maxReach \leftarrow \max(maxReach, i + A[i])$
   b. If $maxReach \geq n$
      Return $jumps + 1$
   c. If $currentReach == i$
      If $maxReach == i$
      Return -1
      Else
      $jumps \leftarrow jumps + 1$
      $currentReach \leftarrow maxReach$
5. Return -1

Smallest Number with Given Digits and Digit Sum

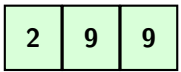
Problem Statement

Given two integers d and s, find the **smallest d-digit positive integer** whose digits sum to exactly s.

The number **cannot start with zero**. Return -1 if no such number exists.

Example 1

$d = 3, \quad s = 20$

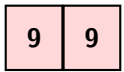


Output: 299

$2 + 9 + 9 = 20$

Example 2

$d = 2, \quad s = 20$



Output: -1

Maximum sum is 18.

Objective

Find the **smallest number** satisfying:

- Exactly d digits.
- Digit sum equals s.
- First digit is nonzero.

Application: Resource Allocation

Distribute a fixed quantity among several positions with limited capacities, while minimizing the allocation to higher-priority positions.

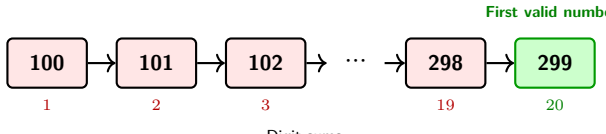
Smallest Number with Given Digits and Digit Sum – Naive Solution

Brute-Force Approach

Enumerate all d-digit numbers in **increasing order**, starting from 10^{d-1}.

For each number, compute its **digit sum**. Return the **first number** whose sum equals s, or -1 if none exists.

Example: $d = 3, \quad s = 20$



Result

The first number whose digits sum to 20 is $\boxed{299}$.

Time Complexity

$$\mathcal{O}(10^d \cdot d)$$

Up to $9 \cdot 10^{d-1}$ candidates, each requiring $\mathcal{O}(d)$ to compute its digit sum.

Space Complexity

$$\mathcal{O}(1)$$

Only the current number and its digit sum are stored.

Smallest Number with Given Digits and Digit Sum – Greedy Solution

Greedy Approach

To obtain the **smallest number**, keep the **leftmost digits as small as possible**.

Reserve 1 for the first digit, then distribute the remaining sum **from right to left**, placing at most 9 in each digit.

Example 1

$d = 3, \quad s = 20$

Start	?	?	?
$s = 19$	?	?	9
$s = 10$	?	9	9
$s = 1$	2	9	9

Output: 299

Example 2

$d = 4, \quad s = 7$

Start	?	?	?	?
$s = 6$	?	?	?	6
$s = 0$	?	0	0	6
Final	1	0	0	6

Output: 1006

Example 3

$d = 2, \quad s = 20$

9	9

$9 + 9 = 18$

$20 > 18$

Impossible!

Output: -1

Feasibility Check

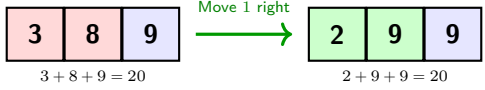
A solution exists if and only if $\boxed{1 \le s \le 9d}$ (assuming $d \ge 1$).

Smallest Number with Given Digits and Digit Sum – Why Greedy Works

Key Observation

A digit's contribution depends on its **position**. Moving one unit from a more significant digit to a less significant digit **decreases the number**, while preserving its digit sum.

Example: Two valid numbers with $d = 3$ and $s = 20$



$$299 < 389$$

Why Does Moving One Unit Right Help?

Moving one unit from position 10^k to position 10^{k-1} changes the number by: $-10^k + 10^{k-1} = -9 \cdot 10^{k-1} < 0$
Thus, we should allocate as much of the digit sum as possible to the **rightmost positions**.

Why Is This Optimal? & Complexity

Optimality: The greedy algorithm fills each rightmost digit up to 9, while reserving at least 1 for the first digit.

Any smaller valid number would require moving some digit sum from a less significant position to a more significant one, which would **increase**, not decrease, the number.

Time: $\boxed{\mathcal{O}(d)}$ Each digit is constructed once. **Auxiliary space:** $\boxed{\mathcal{O}(d)}$

Smallest Number with Given Digits and Digit Sum

Greedy Algorithm

```
Input: int d, int s
Output: Smallest Number with Given Digits and Digit Sum
Instructions:
```

1. If $s < 1$ or $s > 9 \cdot d$
   Return -1
2. $digits \leftarrow$ string containing d '0's
3. $s \leftarrow s - 1$ # Reserve 1 for first digit
4. For i from d down to 2 do
   a. If $s > 9$
      $digits[i] \leftarrow '9'$
      $s \leftarrow s - 9$
   b. Else
      $digits[i] \leftarrow str(s)$
      $s \leftarrow 0$
5. $digits[1] \leftarrow str(1 + s)$
6. Return $digits$

Contents

What is a Greedy Algorithm

Easy Problems

Medium Problems

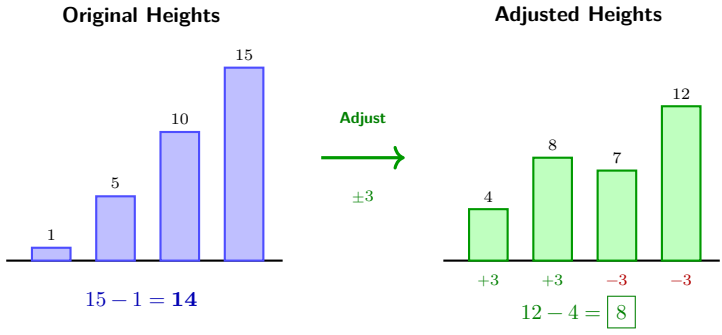
Advanced Problems

Minimize the Maximum Difference Between Heights

Problem Statement

Given n heights A and an integer k, modify **each height exactly once** by either $+k$ or $-k$.
Minimize $\max(A') - \min(A')$, where A' contains the adjusted heights (all must be non-negative).

Example: $A = [1, 5, 10, 15], \quad k = 3$



Real-World Application

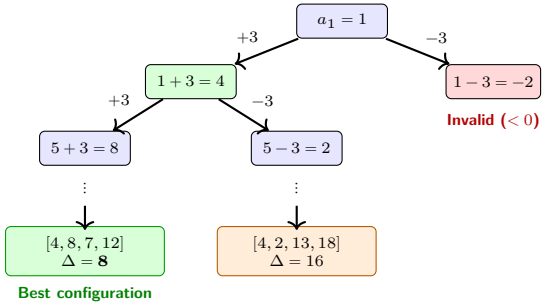
Standardize object heights or product dimensions when each can only be increased or decreased by a fixed amount.

Minimize the Maximum Difference Between Heights — Brute Force

Naive Approach

For each height, try both possible adjustments: $+k$ or $-k$.
Evaluate every valid configuration and keep the smallest difference $\max(A') - \min(A')$.

Example: $A = [1, 5, 10, 15], \quad k = 3$



Complexity

Each of the n heights has 2 choices, giving $\mathbf{2^n}$ configurations. Evaluating one takes $\mathcal{O}(n)$ time.

Time: $\mathcal{O}(n2^n)$	Space: $\mathcal{O}(n)$

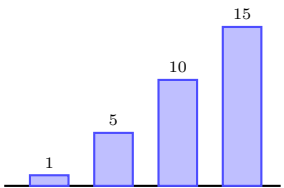
Minimize the Maximum Difference Between Heights — Greedy

Greedy Idea

Sort A. For each splitting point i, increase the first i heights by k and decrease the remaining heights by k. Keep the smallest valid difference.

Example: $A = [1, 15, 10, 5], \quad k = 3 \implies A_{\text{sorted}} = [1, 5, 10, 15] \implies \min_{initial} = 15 - 1 = 14$

Sorted Heights

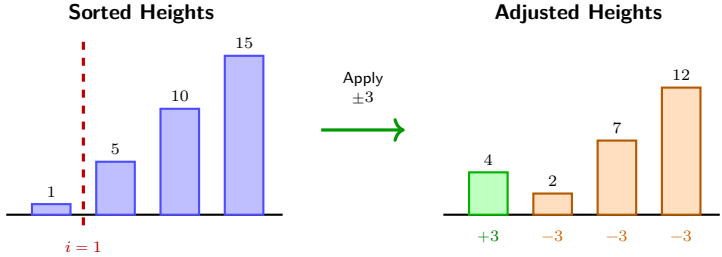


Minimize the Maximum Difference Between Heights — Greedy

Greedy Idea

Sort A. For each splitting point i, increase the first i heights by k and decrease the remaining heights by k. Keep the smallest valid difference.

Example: $A = [1, 15, 10, 5], \quad k = 3 \implies A_{\text{sorted}} = [1, 5, 10, 15] \implies \min_{initial} = 15 - 1 = 14$



Split $i = 1$

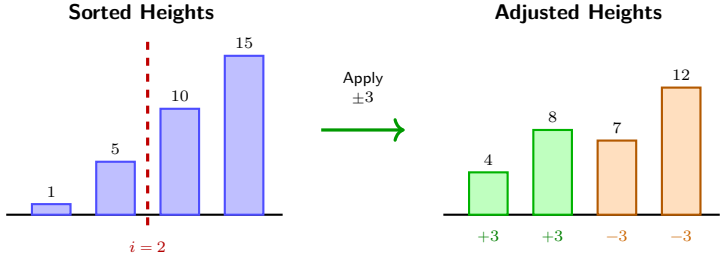
$A' = [4, 2, 7, 12]$ Difference: $12 - 2 = 10$ Best: $\min(14, 10) = \mathbf{10}$

Minimize the Maximum Difference Between Heights — Greedy

Greedy Idea

Sort A. For each splitting point i, increase the first i heights by k and decrease the remaining heights by k. Keep the smallest valid difference.

Example: $A = [1, 15, 10, 5], \quad k = 3 \implies A_{\text{sorted}} = [1, 5, 10, 15] \implies \min_{initial} = 15 - 1 = 14$



Split $i = 2$

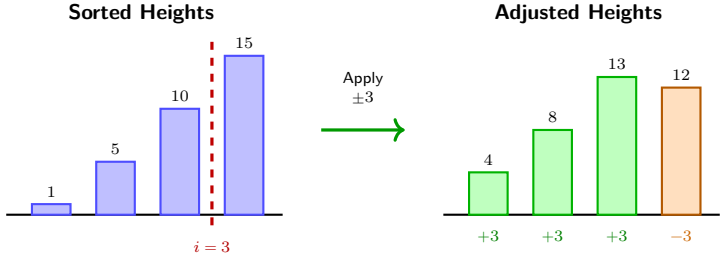
$A' = [4, 8, 7, 12]$ Difference: $12 - 4 = 8$ Best: $\min(10, 8) = \mathbf{8}$

Minimize the Maximum Difference Between Heights — Greedy

Greedy Idea

Sort A. For each splitting point i, increase the first i heights by k and decrease the remaining heights by k. Keep the smallest valid difference.

Example: $A = [1, 15, 10, 5], \quad k = 3 \implies A_{\text{sorted}} = [1, 5, 10, 15] \implies \min_{initial} = 15 - 1 = 14$



Split $i = 3$

$A' = [4, 8, 13, 12]$ Difference: $13 - 4 = 9$ Best: $\min(8, 9) = \mathbf{8}$

Minimize the Maximum Difference Between Heights — Greedy

Evaluating a Splitting Point

After sorting A, consider a split after position i.

$$\text{small} = \min(A[1] + k,\ A[i+1] - k)$$
$$\text{big} = \max(A[i] + k,\ A[n] - k)$$

If $A[i+1] - k < 0$, the split is invalid. Otherwise, minimize big $-$ small.

Example: $A = [1, 5, 10, 15], \quad k = 3$

Split	Adjusted Heights	Difference
$i = 1$	$[4,\ 2,\ 7,\ 12]$	$12 - 2 = 10$
$i = 2$	$[4,\ 8,\ 7,\ 12]$	$12 - 4 = \boxed{8}$
$i = 3$	$[4,\ 8,\ 13,\ 12]$	$13 - 4 = 9$
All $+k$	$[4,\ 8,\ 13,\ 18]$	$18 - 4 = 14$

Complexity

Sorting takes $\mathcal{O}(n \log n)$ time. Each of the $n - 1$ splitting points is evaluated in $\mathcal{O}(1)$ time.

Time: $\mathcal{O}(n \log n)$ Extra Space: $\mathcal{O}(\log n)$

Minimize the Maximum Difference Between Heights — Optimality

Why Are Splitting Points Sufficient?

After sorting, why can we restrict ourselves to **increasing smaller heights** and **decreasing larger heights**?

Consider two heights: $a = 5,\ b = 10,\ k = 3$



General Argument: $a \leq b$

If the smaller height is decreased and the larger increased: $D_{\text{opposite}} = (b + k) - (a - k) = (b - a) + 2k$.
If we reverse the adjustments: $D_{\text{greedy}} = |(b - k) - (a + k)| = |(b - a) - 2k|$.
Since $b - a \geq 0$ and $k \geq 0$:

$$D_{\text{greedy}} \leq D_{\text{opposite}}$$

Moreover, both new values remain inside $[a - k, b + k]$, so the **global range cannot increase**.

Conclusion

Any configuration with a smaller height receiving $-k$ and a larger height receiving $+k$ can be improved or preserved by exchanging their adjustments. Thus, an optimal solution exists with the structure:

$$\underbrace{+k, \ldots, +k}_{\text{smaller heights}} \quad \underbrace{-k, \ldots, -k}_{\text{larger heights}}.$$

Checking all splitting points is therefore sufficient. *The exchange is only a proof technique, not an algorithmic step.*

Minimize the Maximum Difference Between Heights

Greedy Algorithm

Input: Array A of size n, int k
Output: Smallest Maximum Difference Between Heights After Adjustment
Instructions:

1. Sort A in increasing order
2. $res \leftarrow A[n] - A[1]$
3. For i from 1 to $n-1$ do
   a. If $A[i+1] - k < 0$
      continue
   b. $smallest_height \leftarrow min(A[1] + k, A[i+1] - k)$
   c. $biggest_height \leftarrow max(A[i] + k, A[n] - k)$
   d. $res \leftarrow min(res, biggest_height - smallest_height)$
4. Return res

Greedy Algorithms — Session Takeaways

What Is a Greedy Algorithm?

A greedy algorithm builds a solution **step by step**, choosing the **locally best option** at each step.

Key idea: Make decisions without reconsidering previous choices, hoping to achieve a **global optimum**.

When Does Greedy Work?

Two important properties:

- **Greedy-choice property:** a locally optimal choice can be part of a globally optimal solution.
- **Optimal substructure:** the remaining problem can be solved optimally.

Correctness must be proven!
Common techniques: exchange arguments, induction, or contradiction.

Main Takeaway

Before designing a new algorithm, ask whether you can transform the problem into one you already know how to solve.

Why Study Common Problems?

To build a **toolbox of known algorithmic patterns**.

When facing a new problem:

1. Identify its underlying structure.
2. Recognize a **known problem**.
3. Transform it into that problem.
4. Apply its **known solution**.

Recognizing a problem can be more useful than reinventing its solution.

Connection to Complexity Theory

This is related to **problem reductions**:

New Problem $\xrightarrow{\text{reduction}}$ Known Problem

$\Downarrow$ Apply a known algorithm

Reductions also play a central role in studying **P, NP, and NP-completeness**, where they help compare problem difficulty.